

Python'da Gelişmiş Özellikler

NTP II - Hafta 6



Python'da Nesne Tabanlı Programlama: Gelişmiş Özellikler

Bu ders, Python'un nesne tabanlı programlamadaki (OOP) özgün yeteneklerine odaklanmaktadır.

Özel (Dunder) metotların (str, repr, len) işlevselliği incelenecektir.

Statik (@staticmethod) ve Sınıf (@classmethod) metotlarının kullanım alanları analiz edilecektir.

Bu araçların, üniversite düzeyinde daha esnek, okunabilir ve güçlü Python sınıfları oluşturmadaki rolü anlaşılacaktır.



Dunder Metotlar Nedir?

Dunder metotlar (Double Underscore methods) ya da sihirli metotlar, isimleri çift alt çizgi ile başlayıp biten özel metotlardır (örneğin `__init__`, `__str__`). Bu metotlar, Python'un yerleşik operasyonlarını (örneğin toplama, uzunluk hesaplama, yazdırma) sınıf nesnelerimiz için özelleştirmemizi sağlar.

Python'da operatör aşırı yükleme (operator overloading) bu metotlar aracılığıyla gerçekleştirilir.

Bir dunder metot çağırmak yerine, genellikle bu metotları tetikleyen yerleşik bir fonksiyon veya sözdizimi kullanılır (örneğin `len(nesne)` -> `nesne.__len__()`).

Neden Dunder Metotlar Kullanılır?

Kullanım Kolaylığı: Nesnelerin standart Python işlevleriyle uyumlu çalışmasını sağlar (Polimorfizm).

Okunabilirlik: Kodun daha Pythonic (PEP 8 uyumlu) olmasını sağlar.

Protokoller: Python'daki konteynerler (listeler, sözlükler) veya yineleyiciler (iterators) gibi yerleşik davranışları taklit etmemize olanak tanır.

Örnek: 'if nesne in liste:' gibi bir ifadenin çalışması için '__contains__' metodu gereklidir.



`__str__()` ve `__repr__()` Metotlarına Giriş

Bu iki metot, bir nesnenin dizge (string) temsilini döndürmekten sorumludur.

'`__str__`': Nesnenin 'kullanıcı dostu', okunabilir temsilini sağlamalıdır. Genellikle '`print()`' fonksiyonu tarafından çağrılır.

'`__repr__`': Nesnenin 'resmi' dizge temsilini sağlamalıdır. Amaç, nesneyi yeniden oluşturmak için kullanılabilecek kesin bir çıktı sağlamaktır. Geliştiriciler ve hata ayıklama (debugging) için kritik öneme sahiptir.

__str__() Metodunun Kullanım Senaryoları

Bir nesneyi 'print(nesne)' ile doğrudan ekrana yazdırmak istediğimizde, Python otomatik olarak 'nesne.__str__()' metodunu çağırır.

Eğer '__str__' tanımlı değilse, Python bunun yerine '__repr__' metodunu kullanmaya çalışır.

Loglama (logging) sistemlerinde, nesnelerin durumunu hızlıca ve anlaşılır şekilde kaydetmek için idealdir.

'__str__' çıktısının temel amacı, nesnenin mevcut durumunu kısa ve öz bir şekilde özetlemektir.

Örnek Uygulama: Öğrenci Sınıfı için `__str__()`

Amaç: Bir Öğrenci nesnesi print edildiğinde, 'Öğrenci Adı: [Ad], No: [Numara]' gibi anlamlı bir çıktı vermek.

Kod Yapısı:

```
class Öğrenci:
    def __init__(self, ad, no):
        self.ad = ad
        self.no = no
    def __str__(self):
        return f'Öğrenci Adı: {self.ad}, Numara: {self.no}'
```

Kullanım: 'o = Öğrenci('Ali', 123); print(o)' çıktısı, tanımladığımız formatta olacaktır.

__repr__() Metodunun Derinliği

PEP 3109'a göre, '__repr__' çıktısı mümkün olduğunca nesneyi yeniden oluşturmak için gereken geçerli Python ifadesi olmalıdır.

İdeal '__repr__' çıktısı şuna benzer: '<Sınıf Adı(Arg1=Değer1, Arg2=Değer2)>' veya 'SınıfAdı('değer')'.

Hata ayıklama sırasında, liste veya sözlük gibi konteynerleri yazdırdığınızda, içlerindeki nesneler için '__repr__' kullanılır.

Eğer yalnızca bir metot tanımlanacaksa, geliştiriciler genellikle '__repr__' tanımlanmasını ve '__str__'nin onu çağırmasını önerir.

Ogrenci Sınıfı için `__repr__()` Uygulaması

Amaç: Nesneyi yeniden oluşturacak kodu gösteren resmi bir çıktı sağlamak.

Kod Yapısı:

```
class Ogrenci:
```

```
...
```

```
    def __repr__(self):
```

```
        return f'Ogrenci(ad='{self.ad}', no='{self.no})'
```

Kullanım: Python konsolunda sadece nesnenin adını yazmak ('o') veya liste içinde kullanmak ('[o]'), bu metodu tetikler.

Bu çıktı, kopyala-yapıştır ile nesnenin aynısını yaratmaya olanak tanır.

Sınıf Protokolleri: `__len__()` Metodu

'`__len__`' metodu, '`len(nesne)`' yerleşik fonksiyonunu kullanarak nesnemizin 'boyutunu' veya içerdiği öğe sayısını döndürmemizi sağlar. Bu metod, özel sınıfımızı Python'un 'Sıra Protokolü' (Sequence Protocol) veya 'Konteyner Protokolü' ile uyumlu hale getirir. '`__len__`' metodunun ****mutlaka**** negatif olmayan bir tam sayı (integer) döndürmesi gerekir. Aksi takdirde Python hata verir.

__len__() Uygulaması: Kitaplık Sınıfı

Senaryo: Bir Kitaplık sınıfı, içinde birden fazla Kitap nesnesi tutuyor. 'len(kitaplık)' bize kitap sayısını vermelidir.

Kod Örneği:

```
class Kitaplık:  
    def __init__(self, kitaplar=[]):  
        self.kitaplar = kitaplar  
    def __len__(self):  
        return len(self.kitaplar)
```

Kullanım: 'k = Kitaplık([...]); print(len(k))' artık içerideki kitap listesinin uzunluğunu doğru bir şekilde döndürür.

Diğer Önemli Dunder Metotlar (Kısa Bakış)

- '__init__(self, ...)': Nesnenin oluşturulmasından hemen sonra ilklendirilmesi.
- '__new__(cls, ...)': Nesnenin bellekte oluşturulması (daha nadir kullanılır, genellikle metaclass'ler ile).
- '__getitem__(self, key)': Köşeli parantez erişimi (örneğin nesne).
- '__add__(self, other)': '+' operatörünü aşırı yükleme.
- '__call__(self, ...)': Nesneyi bir fonksiyon gibi çağırabilme (örneğin nesne()).

Metot Türleri: Statik ve Sınıf Metotları

Python, standart örnek metotlarının (instance methods) yanı sıra, özel amaçlar için tasarlanmış iki metot türü sunar: Statik Metotlar ve Sınıf Metotları. Örnek metotları, 'self' argümanını alır ve bir nesnenin durumuna erişebilir. Bu özel metotlar ise, ya sınıfın kendisiyle ('cls') ya da hiçbir özel referans olmadan çalışır. Bu ayırım, sınıf tasarımında sorumlulukları netleştirmeye yardımcı olur.

Statik Metotlar (@staticmethod)

Tanım: Statik metotlar, bir sınıfa ait olsalar bile, sınıfın veya nesnenin hiçbir durumuna (state) erişim gerektirmeyen işlevlerdir.

Anahtar Özellik: Ne 'self' (örnek) ne de 'cls' (sınıf) argümanını otomatik olarak almazlar.

Kullanım Alanı: Sınıfla mantıksal olarak ilişkili olan, ancak sınıf verilerini kullanmayan yardımcı (utility) fonksiyonlar veya hesaplamalar için idealdir. Statik metotlar, esasen normal bir fonksiyondur, ancak organizasyon amacıyla sınıf içine yerleştirilmiştir.

Statik Metotların Uygulanması

Statik metotlar, '@staticmethod' dekoratörü kullanılarak tanımlanır.
Erişim: Statik metotlar, hem sınıf üzerinden (Örn: SınıfAdı.statik_metot()) hem de sınıfın bir örneği üzerinden (Örn: nesne.statik_metot()) çağrılabilir.
Örnek: Bir 'Matematik' sınıfında 'topla(a, b)' veya bir 'Yapılandırma' sınıfında 'varsayılan_ayarları_oku()' fonksiyonları statik olabilir.



Statik Metot Kullanım Alanları (Detaylı)

Veri Doğrulama: Sınıfın `__init__` metoduna girmeden önce dışarıdan gelen veriyi kontrol eden metotlar.

Formatlama: Belirli bir çıktı formatı oluşturan veya girdi formatını dönüştüren araçlar.

Bağımsız Hesaplamalar: Sınıfın yapısıyla değil, genel bir kural veya formülle ilgili olan hesaplamalar (örneğin tarih dönüşümleri).

Statik metotlar, miras (inheritance) durumunda davranışlarını değiştirmezler.

Sınıf Metotları (@classmethod)

Tanım: Sınıf metotları, sınıfın kendisi üzerinde işlem yapmak için kullanılır. Örnek nesnesi üzerinde değil, doğrudan sınıfın durumu üzerinde çalışır. Anahtar Özellik: İlk argüman olarak otomatikman sınıfın kendisini ('cls') alır. Bu argüman sayesinde sınıf değişkenlerine erişilebilir veya sınıfın yeni bir örneği oluşturulabilir. Kullanım Alanı: Alternatif yapıcılar (factory methods) oluşturmak ve sınıfa özgü ayarları değiştirmek için kullanılır.



Sınıf Metotlarının Uygulanması

Sınıf metotları, '@classmethod' dekoratörü kullanılarak tanımlanır.

'cls' Argümanı: Bu argüman sayesinde metot, hangi sınıftan çağrıldığını bilir. Bu özellik, özellikle miras alma durumlarında dinamik örnek oluşturma yeteneği sağlar.

Factory Metodu: En yaygın kullanımı, sınıfı farklı girdi formatlarından (örneğin JSON, CSV) başlatmak için alternatif yapıcılar sağlamaktır.

Sınıf Metotları: Alternatif Yapıcılar

Standart yapıcı `'__init__'`, genellikle temel parametrelerle çalışır. Ancak verinin farklı formatta gelmesi gerekebilir.

Örnek: 'Tarih' sınıfının normalde yıl, ay, gün ile ilklendirildiğini varsayalım.

Alternatif Yapıcı: 'Tarih.from_string('2023-10-27')' gibi bir sınıf metodu, dizgeyi ayrıştırarak `'cls(yıl, ay, gün)'` çağırısı yapabilir.

Bu, `'__init__'` metodunu karmaşıklaştırmadan esneklik sağlar.

Miras ve Sınıf Metotları İlişkisi

Sınıf metotları, miras alınan sınıflarda önemlidir.

Eğer bir ana sınıfın sınıf metodu, yeni bir nesne döndürüyorsa ('return cls(...)'), miras alan alt sınıf bu metodu çağırdığında, döndürülen nesne alt sınıf tipinde olacaktır.

Bu, polimorfik nesne oluşturma (oluşturulan nesnenin tipinin çağrıyı yapan sınıfa bağlı olması) sağlar.

Karar Verme Matrisi: Hangi Metodu Seçmeli?

Örnek Metotları: Nesnenin durumuna (self.veri) erişim gerektiğinde kullanılır.
Sınıf Metotları (@classmethod): Sınıfın kendisine (cls) ve sınıf değişkenlerine erişim gerektiğinde veya alternatif yapıcı gerektiğinde kullanılır.
Statik Metotlar (@staticmethod): Ne nesnenin ne de sınıfın durumuna ihtiyaç duyan, yalnızca mantıksal olarak sınıfla ilişkili olan yardımcı fonksiyonlar için kullanılır.

Dunder Metotlar İle Python Protokolleri

Python, 'Duck Typing' (Ördek Tiplemesi) ilkesini yoğun olarak kullanır: Eğer bir şey ördek gibi yürüyüp, ördek gibi vaklıyorsa, o bir ördektir.

Protokoller: Dunder metotlar, nesnelerin belirli davranışları sergilemesini sağlayan protokolleri tanımlar.

Örneğin, '`__iter__`' ve '`__next__`' metotlarını uygulayan herhangi bir nesne, bir yineleyici (iterator) protokolüne uyar ve 'for' döngülerinde kullanılabilir.

Özel Operatör Aşırı Yükleme Dunder Metotları

Python'daki tüm operatörler dunder metotlara karşılık gelir:

Toplama: `'__add__'` ($a + b$)

Çıkarma: `'__sub__'` ($a - b$)

Eşitlik Kontrolü: `'__eq__'` ($a == b$)

Küçüktür/Büyüktür: `'__lt__'`, `'__gt__'`

Bu metotlar, kendi özel veri tiplerimizin yerleşik tipler (int, float) gibi davranmasını sağlar.

Örnek: Vektör Sınıfında Toplama İşlemi

```
class Vektor:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y  
    def __add__(self, other):  
        return Vektor(self.x + other.x, self.y + other.y)
```

Kullanım: 'v1 = Vektor(1, 2); v2 = Vektor(3, 4); v3 = v1 + v2' artık v3'ü Vektor(4, 6) olarak tanımlayacaktır.

Bu, kodun matematiksel olarak sezgisel kalmasını sağlar.

Bellek Yönetimi ve `__del__()` Metodu

'`__del__`' metodu (finalizer), bir nesnenin referans sayımı sıfıra düştüğünde ve nesne bellekten kaldırılmadan hemen önce çağrılır.

Önemli Uyarı: Bu metod, kaynak serbest bırakma (dosya kapatma, ağ bağlantısı kesme) için ****güvenilir bir mekanizma değildir****.

Çünkü Python'un çöp toplayıcısının ne zaman çalışacağı garantilenemez. Kaynak yönetimi için genellikle 'with' ifadesi ile kullanılan '`__enter__`' ve '`__exit__`' (Context Manager) protokolü tercih edilmelidir.

Context Manager Protokolü

Context Manager'lar, bir kaynakla ilgili işlemlerin başında ve sonunda gerçekleştirilmesi gereken kurulum ve temizleme eylemlerini otomatikleştirir.

'__enter__': with bloğuna girerken çağrılır, genellikle kaynağı açar ve döndürür.

'__exit__': with bloğundan çıkarken çağrılır (bir hata olsa bile), kaynağı temizler veya kapatır.

Dosya işlemleri ('with open(...)') bu protokolün en yaygın örneğidir.

Statik Metotların Sınıf İçi Veri Erişim Kısıtlaması

Statik metotlar, bir sınıftaki örneklerle özgü veriye (self.veri) doğrudan erişemez.

Eğer bir statik metot bu verilere ihtiyaçot: Devralındığında, davranışını korur. Alt sınıfta çağrıldığında bile ana sınıftaki orijinal mantığı çalıştırır, çünkü 'cls"ye erişimi yoktur.

Sınıf Metodu: Devralındığında, 'cls' argümanı sayesinde alt sınıfın referansını alır. Bu, alt sınıfa özgü yeni nesneler oluşturulmasına olanak tanır (Polimorfik Fabrika Metodu).



Güvenlik Perspektifi: Kapsülleme ve Dunder Metotlar

Python'da katı bir özel (private) erişim kısıtlayıcısı yoktur. Kapsülleme, adlandırma kurallarıyla sağlanır.

Tek alt çizgi ('_'): Geliştiriciye bu ögenin iç kullanım için olduğunu ve dışarıdan doğrudan erişilmemesi gerektiğini bildirir.

Çift alt çizgi ('__'): 'Name mangling' (isim karmaşıklaştırma) tetikler. Python, bu öge adını '_SınıfAdı__isim' şeklinde değiştirir.

Dunder metotlar, bu kuralların dışındadır; onlar, Python sistemi için genel, iyi tanımlanmış arayüzlerdir.

Statik Metotların Test Edilebilirliği

Statik metotlar, sınıfın veya nesnenin durumuna bağımlı olmadıkları için genellikle en kolay test edilebilen metotlardır. Birim testi (unit test) yazarken, bir sınıf örneği oluşturmaya gerek kalmaz. Bu, test kurulumunu (setup) basitleştirir ve testlerin daha hızlı çalışmasını sağlar.



Sınıf Metotlarının Mirasta Rolü

Bir ana sınıf, temel veriyi işleyen bir sınıf metodu tanımladığında, alt sınıflar bu metodu miras alabilir.

Alt sınıf bu metodu çağırdığında, 'cls' referansı alt sınıfı işaret ettiği için, metot alt sınıfa ait nesneler üretecektir.

Bu, aynı fabrika metodunu kullanarak farklı tipte nesneler üretmek anlamına gelir.



__str__ ve __repr__ için En İyi Uygulamalar

Her zaman '`__repr__`' tanımlayın. Geliştirici araçları (örneğin hata ayıklayıcılar) için kritik öneme sahiptir.

Eğer '`__str__`' tanımlı değilse, '`print()`' otomatik olarak '`__repr__`'i kullanır.

'`__str__`' çıktısı, kullanıcıya yönelik olmalı, kısa ve bilgilendirici olmalıdır.

'`__repr__`' çıktısı, mümkünse nesneyi yeniden oluşturabilecek formatta olmalıdır.

Konteyner Dunder Metotları: `__getitem__`

'`__getitem__`(self, key)', nesnelerin köşeli parantez '[' ile erişilebilir olmasını sağlar (örneğin 'nesne').

Bu, özel bir veri yapısını liste veya sözlük gibi kullanmamıza olanak tanır.

Eğer dizin (index) geçerli değilse 'IndexError' veya anahtar (key) geçerli değilse 'KeyError' yükseltmek bu metodun sorumluluğundadır.

Sıra Protokolü (Sequence Protocol) Özeti

Bir nesnenin tam bir sıra (sequence) gibi davranması için en azından `'__len__'` ve `'__getitem__'` metotlarını uygulaması gerekir. Bu, nesnemizin Python'un yerleşik listeleri, demetleri (tuples) ve dizgeleri gibi davranabileceği anlamına gelir. Bu protokollerin uygulanması, kodun genel Python ekosistemiyle uyumunu artırır.

Statik Metotların Sınıf Dışı Alternatifleri

Statik metotlar, genellikle yalnızca organizasyon amacıyla kullanılır. Eğer bir fonksiyonun sınıf veya nesneyle bağı yoksa, sınıfın içinde '@staticmethod' olarak tanımlamak yerine, doğrudan modül düzeyinde (dosyanın en üstünde) tanımlanması daha temiz bir çözüm olabilir. Bu, sınıf arayüzünü gereksiz işlevlerle doldurmayı engeller.



Dunder Metotlar ve Güvenlik Açıkları

Özel metotların aşırı yüklenmesi, özellikle kullanıcı girdisiyle etkileşime girildiğinde dikkatli olunmasını gerektirir.

Örneğin, `'__init__'` veya `'__setattr__'` metotları, kontrolsüz veri atamalarına izin verirse güvenlik zafiyetleri (örneğin SQL injection benzeri nesne manipülasyonu) yaratabilir.

Güvenli kodlama, dunder metotları kullanırken her zaman girdi temizliğini (input sanitization) zorunlu kılar.



Nesne Tabanlı Programlamada Tasarım İlkeleri

Tek Sorumluluk ilkesi (SRP): Bir sınıfın yalnızca tek bir sorumluluğu olmalıdır. Statik metotlar, yardımcı işlevleri ayırarak bu ilkeye uymayı kolaylaştırır.

Açık/Kapalı ilkesi (OCP): Sınıflar, uzatmaya açık, ancak değiştirmeye kapalı olmalıdır.

Dunder metotlar, nesnelerin dış arayüzünü sabit tutarken iç davranışlarını değiştirmemizi sağlar.

Örnek Senaryo: Veri Yapılandırma Sınıfı

Sınıfımız, yapılandırma ayarlarını tutsun ve bunları JSON dosyasından yükleyebilsin.

Statik Metot: `'is_valid_json(data)'` - Gelen ham verinin formatını kontrol eder (Nesne durumundan bağımsız).

Sınıf Metodu: `'from_json_file(filename)'` - Dosyayı okur, statik metotla doğrular ve `'cls(...)'` ile yeni bir örnek oluşturur (Alternatif yapıcı).

__getattr__ ve Dinamik Özellik Erişimi

'__getattr__(self, name)' metodu, erişilmeye çalışılan özellik (attribute) normal yollarla bulunamadığında çağrılır.

Bu, dinamik olarak özellik oluşturmak veya eksik özellik erişimini yönetmek için kullanılır.

Güvenlik Riski: Yanlış kullanıldığında, programın davranışını öngörülemez hale getirebilir veya yetkisiz erişime yol açabilir.



Python'daki Enum Yapıları

Python'da sabit değer setlerini (constants) tanımlamak için 'enum.Enum' kullanılır.

Enum'lar, sınıf değişkenlerini ve statik yapıları organize etmenin daha güvenli ve okunaklı bir yoludur.

Enum metotları, genellikle sınıf metotları veya statik metotlar olarak tanımlanır.



Gelişmiş Tip İpuçları ve Dunder Metotlar

Python 3.5 ve sonrası ile gelen tip ipuçları (type hints), dunder metotlar dahil olmak üzere tüm metotlar için kullanılmalıdır.

Bu, statik kod analiz araçlarının ('mypy' gibi) nesne protokollerinin doğru uygulanıp uygulanmadığını kontrol etmesine olanak tanır (örneğin, '__len__'in int döndürüp döndürmediğini kontrol etme).

Statik ve Sınıf Metotları: Kod Tekrarını Azaltma

Bu metot türleri, birbiriyle ilişkili işlevleri tek bir yerde toplayarak kod tekrarını önler.

Statik metotlar, farklı sınıflarda tekrar eden yardımcı fonksiyonların tek bir yerde toplanmasını sağlayabilir.

Sınıf metotları, farklı yapıcı mantıklarını '__init__' dışına taşıyarak '__init__'in sadece temel ilklendirmeye odaklanmasını sağlar.



Python Güvenliği ve Nesne Temsili

Hassas Veri Riski: `'__str__'` veya `'__repr__'` metotları, loglama veya hata ayıklama sırasında nesnenin içeriğini açığa çıkarır.

Eğer bir 'Ogrenci' sınıfının `'__repr__'` çıktısı gizli şifre veya kimlik numarası içeriyorsa, bu veriler log dosyalarına sızabilir.

Tavsiye: Temsil metotları, yalnızca güvenli ve tanımlayıcı (non-sensitive) verileri içermelidir (örneğin, sadece nesne ID'si veya adı).

Python'da Nesne Tabanlı Programlama Özeti

Dunder Metotlar: Python ekosistemiyle uyumlu çalışan güçlü ve sezgisel nesneler oluşturmanın temelidir.

'__str__' ve '__repr__' nesne çıktısını kontrol eder.

'__len__' nesneleri boyutlandırılabilir yapar.

Statik Metotlar: Nesne veya sınıf durumundan bağımsız yardımcı işlevler sunar.

Sınıf Metotları: Alternatif yapıcılar ve sınıf düzeyinde işlemler için 'cls' referansını kullanır.

İleri Okuma ve Kaynaklar

Fluent Python: Python'un dunder metotları ve veri modelleri hakkında derinlemesine bilgi sağlar.

Python Belgeleri: Veri Modeli bölümü (Data Model) tüm dunder metotları resmi olarak listeler.

PEP 8 ve PEP 20 (Zen of Python): Pythonic kod yazım kurallarını anlamak.



Dunder metotların bir nesnenin davranışını nasıl temelden değiştirdiğini özetleyiniz.

Statik, sınıf ve örnek metotları arasındaki temel farklar nelerdir?

Öğrenci sınıfı örneğinde `'__str__'` metodunu neden kullanırız?

'len()' fonksiyonunu kendi sınıfımızda kullanabilmek için hangi dunder metot gereklidir?



Ek: Operator Aşırı Yüklemede Refleksiyon

Normalde, ' $a + b$ ' işlemi ' $a.__add__(b)$ ' olarak çağrılır. Ancak eğer ' a ' metodu uygulamazsa veya 'NotImplemented' döndürürse, Python ' $b.__radd__(a)$ ' metodunu aramaya başlar (Refleksiyonel toplama). Bu metotlar (' $__radd__$ ', ' $__rsub__$ ', vb.), karışık tip işlemlerinde (örneğin $int + MyClass$) uyumluluğu sağlamak için kritik öneme sahiptir.

Python'da Gelişmiş Özellikler

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

